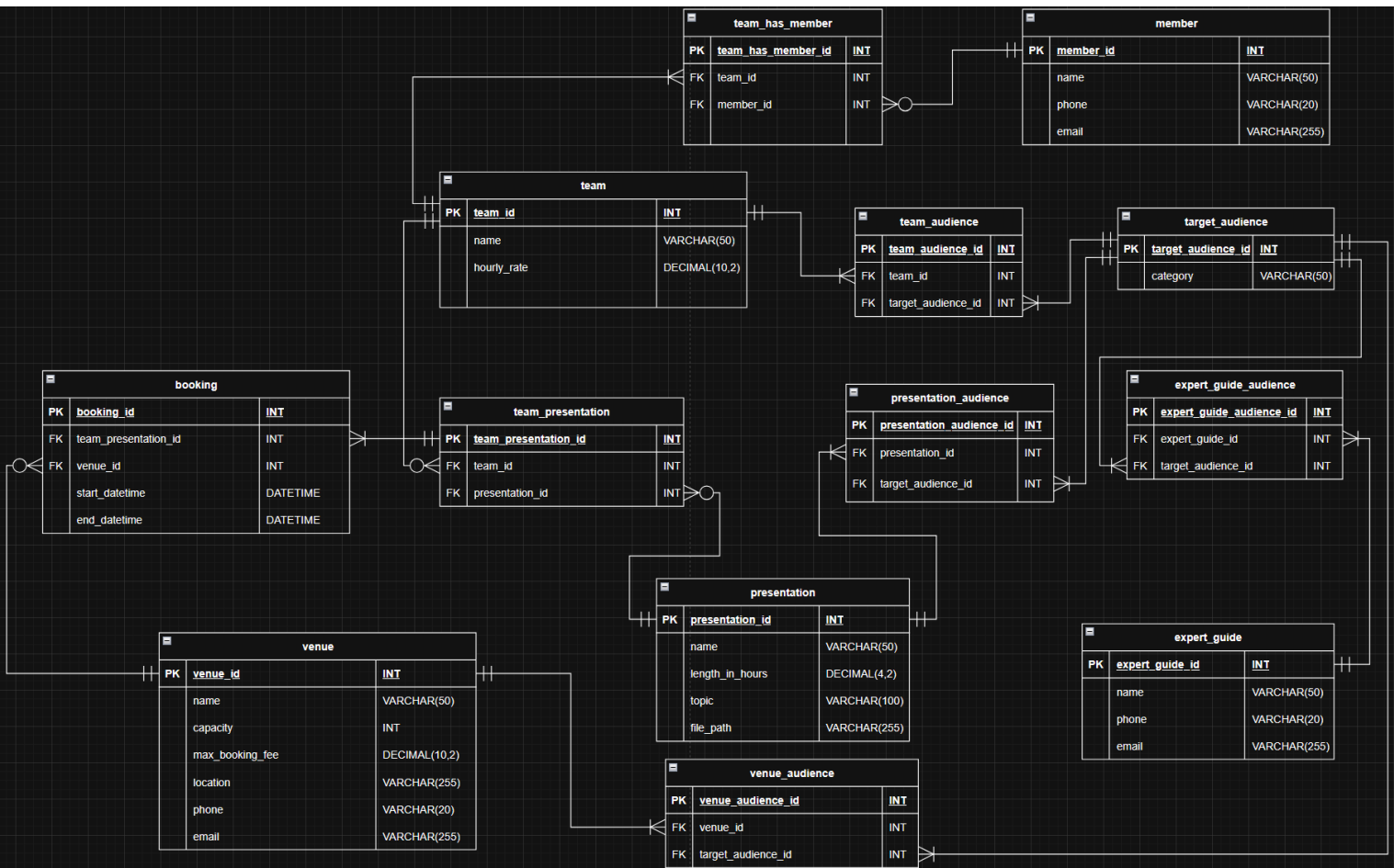


Design and build of a database application

Table of Contents

ERD diagram:.....	1
Python code:.....	2
add_teams.html.....	12
base.html.....	14
booking_costs_from_team.html.....	18
booking_costs_from_venue.html.....	20
home.html.....	23
non_solo_teams.html.....	23
suitable_team_for_target_audience.html.....	24
suitable_team_for_venue.html.....	26
teams_with_one_solo_member.html.....	28
venue_with_3_or_more_target_audiences.html.....	29
assessment_1_database.sql.....	30
assesment_1_database_adding data.sql.....	40
Example commands slightly edited to work in MYSQL workbench and shown in video.....	53
part_1.sql.....	53
part_2.sql.....	54
part_3.sql.....	55
part_4.sql.....	55
part_5.sql.....	56
part_6.sql.....	57
part_7.sql.....	58
part_8.sql.....	59
part_9.sql.....	60
part_10.sql.....	60

ERD diagram:



Python code:

```
from flask import Flask, render_template, request
import mysql.connector
```

```
app = Flask(__name__)
```

```
def get_database_connection():
    return mysql.connector.connect(
        host='localhost',
        user='root',
        password='mysql',
        database='booking_system'
    )
```

```
#home
@app.route('/')
def home():
    return render_template('home.html')
#home
```

#A list of teams whose target audiences are suitable to perform at a chosen venue (return at least 2 teams). Choose the target audiences and a suitable venue.
#part 1

```
@app.route('/suitable_team_for_venue', methods=['GET', 'POST'])
def suitable_team_for_venue():
    conn = get_database_connection()
    cursor = conn.cursor(dictionary=True)

    #fetching dropdown list
    cursor.execute("SELECT venue_id, name FROM venue")
    fetched_dropdown_list = cursor.fetchall()

    result = []
    selected_row = None

    #search button
    if request.method == 'POST':
        get_row = request.form.get('venue_id')

        #sends query to database
        cursor.execute("SELECT name FROM venue WHERE venue_id =
%s", (get_row,))
        record = cursor.fetchone()

        if record:
            selected_row = (record['name'])

    query = """
        SELECT DISTINCT
            team.name,
            team.hourly_rate,
            target_audience.category
        FROM team
        JOIN team_audience
            ON team.team_id = team_audience.team_id
        JOIN target_audience
            ON team_audience.target_audience_id =
target_audience.target_audience_id
        JOIN venue_audience
```

```

        ON target_audience.target_audience_id =
venue_audience.target_audience_id
        WHERE venue_audience.venue_id = %s
"""

```

```

        cursor.execute(query, (get_row,))
        result = cursor.fetchall()

```

```

    cursor.close()
    conn.close()

```

```

    return render_template(
        'suitable_team_for_venue.html',
        venues = fetched_dropdown_list,
        teams = result,
        selected_venue = selected_row
    )

```

```

#####
#####

```

#A list of teams whose target audiences are suitable to perform at a chosen venue (return at least 2 teams). Choose the target audiences and a suitable venue.

#part 2

```

@app.route('/suitable_team_for_target_audience', methods=['GET', 'POST'])

```

```

def suitable_team_for_target_audience():
    conn = get_database_connection()
    cursor = conn.cursor(dictionary=True)

```

```

    #fetching dropdown list
    cursor.execute("SELECT target_audience_id, category FROM
target_audience")
    fetched_dropdown_list = cursor.fetchall()

```

```

    result = []
    selected_row = None

```

```

    #search button
    if request.method == 'POST':
        get_row = request.form.get('target_audience_id')

```

```

    #sends query to database
    cursor.execute("SELECT category FROM target_audience WHERE
target_audience_id = %s", (get_row,))
    record = cursor.fetchone()

```

```

    if record:

```

```
selected_row = record['category']
```

```

query = """
    SELECT DISTINCT
        team.name AS team_name,
        team.hourly_rate,
        target_audience.category,
        venue.name AS venue_name
    FROM team
    JOIN team_audience
        ON team.team_id = team_audience.team_id
    JOIN target_audience
        ON team_audience.target_audience_id =
target_audience.target_audience_id
    JOIN venue_audience
        ON target_audience.target_audience_id =
venue_audience.target_audience_id
    JOIN venue
        ON venue_audience.venue_id = venue.venue_ID
    WHERE target_audience.target_audience_id = %s;

    """

    cursor.execute(query, (get_row,))
    result = cursor.fetchall()

    cursor.close()
    conn.close()

    return render_template(
        'suitable_team_for_target_audience.html',
        audiences = fetched_dropdown_list,
        teams = result,
        selected_venue = selected_row
    )
#####
#####

```

Output at least 2 teams that have just one solo member (must include member's name) in the database.

```

@app.route('/teams_with_one_solo_member', methods=['GET', 'POST'])
def teams_with_one_solo_member():
    conn = get_database_connection()
    cursor = conn.cursor(dictionary=True)

```

```

query = """
    SELECT
        team.name AS team_name,
        MAX(member.name) AS member_name
    FROM team
    JOIN team_has_member
        ON team.team_id = team_has_member.team_id
    JOIN member
        ON member.member_id = team_has_member.member_id
    GROUP BY team.team_id, team.name
    HAVING COUNT(team_has_member.member_id) = 1;
"""

cursor.execute(query)
result = cursor.fetchall()

cursor.close()
conn.close()

```

```

return render_template(
    'teams_with_one_solo_member.html',
    teams = result
)

```

```

#####
#####

```

#A list of venues that host teams from 3 or more target audiences
(return at least 2 venues)

```

@app.route('/venue_with_3_or_more_target_audiences',
methods=['GET', 'POST'])

```

```

def venue_with_3_or_more_targe_audiences():
    conn = get_database_connection()
    cursor = conn.cursor(dictionary=True)

```

```

    query = """
        SELECT
            venue.name AS venue_name,
            COUNT(DISTINCT target_audience.target_audience_id) AS
audience_count,
            GROUP_CONCAT(DISTINCT target_audience.category
SEPARATOR ', ') AS audience_list
        FROM venue
        JOIN venue_audience ON venue.venue_id =
venue_audience.venue_id
        JOIN target_audience ON venue_audience.target_audience_id
= target_audience.target_audience_id
    """

```

```

        GROUP BY venue.venue_id, venue.name
        HAVING audience_count >= 3;
"""
cursor.execute(query)
result = cursor.fetchall()

cursor.close()
conn.close()

return render_template(
    'venue_with_3_or_more_target_audiences.html',
    venues = result
)

```

```

#####
#####

```

#create a query to show all the members of 2 non-solo teams,
showing clearly which team each member belongs to

```

@app.route('/non_solo_teams')
def non_solo_teams():
    conn = get_database_connection()
    cursor = conn.cursor(dictionary=True)

    query = """
        SELECT
            team.name AS team_name,
            member.name AS member_name
        FROM team
        JOIN team_has_member ON team.team_id =
team_has_member.team_id
        JOIN member ON team_has_member.member_id =
member.member_id
        WHERE team.team_id IN (
            SELECT team_id
            FROM team_has_member
            GROUP BY team_id
            HAVING COUNT(member_id) > 1
        )
        ORDER BY team.name;
    """

    cursor.execute(query)
    results = cursor.fetchall()

```

```

cursor.close()
conn.close()

```

```

return render_template(
    'non_solo_teams.html',
    team_members=results
)

```

```

#####
#####

```

#A booking has a start time and end time, and each team has a fee in £ per hour. Write a query in Python to
 #run a calculated field of the total event cost (as
 TOTAL_BOOKING_COST) for a team at a venue. Flag any potential
 #booking that is going to exceed the venue maximum booking fee.
 (You can chose the team and the venue.)

```

@app.route('/booking_costs_from_team', methods=['GET', 'POST'])
def calculate_booking_costs_team():
    connection = get_database_connection()
    cursor = connection.cursor(dictionary=True)

    #fetching dropdown list
    cursor.execute("SELECT team_id, name FROM team")
    teams_list = cursor.fetchall()

    results = []
    selected_name = None

    #search button
    if request.method == 'POST':
        chosen_id = request.form.get('team_selection_id')

        #sends query to database
        cursor.execute("SELECT name FROM team WHERE team_id = %s",
(chosen_id,))
        team_record = cursor.fetchone()
        if team_record:
            selected_name = team_record['name']

        #brief wanted in this format (dd:mm:yyyy hh:mm:ss) this is
my workaround
        correcting_datetime_format = '%d/%m/%Y %H:%i:%s'

        query = """
        SELECT
            venue.name AS venue_name,
            venue.max_booking_fee AS venue_max_fee,

```

```

        team.name AS TEAM_NAME,
        team.hourly_rate AS team_hourly_rate,
        DATE_FORMAT(booking.start_datetime, %s) AS
start_time,
        DATE_FORMAT(booking.end_datetime, %s) AS end_time,
        TIMESTAMPDIFF(HOUR, booking.start_datetime,
booking.end_datetime) AS duration_in_hours,
        (TIMESTAMPDIFF(HOUR, booking.start_datetime,
booking.end_datetime) * team.hourly_rate) AS total_booking_cost,
        CASE
            WHEN (TIMESTAMPDIFF(HOUR,
booking.start_datetime, booking.end_datetime) * team.hourly_rate)
> venue.max_booking_fee
            THEN 'OVER BUDGET'
            ELSE 'WITHIN BUDGET'
        END AS booking_status
    FROM booking
    JOIN venue ON booking.venue_id = venue.venue_id
    JOIN team_presentation ON booking.team_presentation_id
= team_presentation.team_presentation_id
    JOIN team ON team_presentation.team_id = team.team_id
    WHERE team.team_id = %s
    """

```

```

        cursor.execute(query, (correcting_datetime_format,
correcting_datetime_format, chosen_id))
        results = cursor.fetchall()

```

```

    cursor.close()
    connection.close()

```

```

    return render_template(
        'booking_costs_from_team.html',
        teams_dropdown = teams_list,
        bookings = results,
        selected_name = selected_name
    )

```

```

#####
#####

```

#A booking has a start time and end time, and each team has a fee in £ per hour. Write a query in Python to #run a calculated field of the total event cost (as TOTAL_BOOKING_COST) for a team at a venue. Flag any potential #booking that is going to exceed the venue maximum booking fee. (You can chose the team and the venue.)

```

@app.route('/booking_costs_from_venue', methods=['GET', 'POST'])

```

```

def calculate_booking_costs_venue():
    connection = get_database_connection()
    cursor = connection.cursor(dictionary=True)

    # fetching dropdown list
    cursor.execute("SELECT venue_id, name FROM venue")
    fetched_dropdown_list = cursor.fetchall()

    results = []
    selected_name = None

    # search button
    if request.method == 'POST':
        get_row = request.form.get('venue_selection_id')

        # sends query to database to get venue name
        cursor.execute("SELECT name FROM venue WHERE venue_id =
%s", (get_row,))
        record = cursor.fetchone()
        if record:
            selected_name = record['name']

        # Date format workaround
        correcting_datetime_format = '%d/%m/%Y %H:%i:%s'

        query = """
            SELECT
                team.name AS team_name,
                team.hourly_rate AS team_hourly_rate,
                DATE_FORMAT(booking.start_datetime, %s) AS
start_time,
                DATE_FORMAT(booking.end_datetime, %s) AS end_time,
                venue.max_booking_fee AS venue_max_fee,
                TIMESTAMPDIFF(HOUR, booking.start_datetime,
booking.end_datetime) AS duration_in_hours,
                (TIMESTAMPDIFF(HOUR, booking.start_datetime,
booking.end_datetime) * team.hourly_rate) AS total_booking_cost,
                CASE
                    WHEN (TIMESTAMPDIFF(HOUR,
booking.start_datetime, booking.end_datetime) * team.hourly_rate)
> venue.max_booking_fee
                    THEN 'OVER BUDGET'
                    ELSE 'WITHIN BUDGET'
                END AS booking_status
            FROM booking
            JOIN venue ON booking.venue_id = venue.venue_id
            JOIN team_presentation ON booking.team_presentation_id
= team_presentation.team_presentation_id
            JOIN team ON team_presentation.team_id = team.team_id
            WHERE booking.venue_id = %s
        """

```

```

        # Passing the format string twice for start and end times,
        then the venue ID
        cursor.execute(query, (correcting_datetime_format,
                                correcting_datetime_format, get_row))
        results = cursor.fetchall()

    cursor.close()
    connection.close()

    return render_template(
        'booking_costs_from_venue.html',
        venues_dropdown = fetched_dropdown_list,
        bookings = results,
        selected_name = selected_name
    )

```

```

#####
#####

```

```

#Create a MySQL INSERT INTO query in Python, operated from the
main menu. It needs to add a further 2 teams
#from the user's input, and add at least 2 members each per team
(also from user input), ensuring any relevant
#connected tables are updated too.

```

```

@app.route('/add_teams', methods=['GET', 'POST'])
def add_teams():
    message = None

    if request.method == 'POST':
        conn = get_database_connection()
        cursor = conn.cursor()

        try:

            # inserting team
            insert_team_name = request.form['table_name']
            insert_hourly_rate = request.form['table_rate']
            cursor.execute("INSERT INTO team (name, hourly_rate)
VALUES (%s, %s)", (insert_team_name, insert_hourly_rate))

            # get new row id
            team_id = cursor.lastrowid

```

```

        # insert team members block
        table_members = [request.form['table_members_1'],
request.form['table_members_2']]
        for member_name in table_members:

            # add new member to table
            cursor.execute("INSERT INTO member (name) VALUES
(%s)", (member_name,))
            member_id = cursor.lastrowid

            # filling junction table(team_has_member) to
connect to team
            cursor.execute("INSERT INTO team_has_member
(team_id, member_id) VALUES (%s, %s)", (team_id, member_id))

            conn.commit()
            message = "Successfully committed 1 teams and 2 members
to the booking system"

            # rollback if there is an error
            except Exception as e:
                conn.rollback()
                message = f"An error occurred: {e}"

            finally:
                cursor.close()
                conn.close()

        return render_template('add_teams.html', message=message)

```

#end of program dont delete this bit

```

if __name__ == '__main__':
    app.run(debug=True)

```

add_teams.html

```

{% extends "base.html" %}

{% block title %}Add New Teams & Members{% endblock %}

{% block content %}
    <h1>Add New Teams & Members</h1>

    {% if message %}
        <div style="padding: 15px; background-color: #d4edda;
color: #155724; border: 1px solid #c3e6cb; margin-bottom: 20px;">
            <strong>{{ message }}</strong>
        </div>
    {% endif %}

    <form method="POST" action="/add_teams">

        <fieldset style="margin-bottom: 20px; padding: 15px;
border: 2px solid #ddd;">
            <legend><strong>Adding Team</strong></legend>

            <label>Team Name:</label><br>
            <input type="text" name="table_name" required
placeholder="e.g. The Paul Jackson team"><br><br>

            <label>Hourly Rate (£):</label><br>
            <input type="number" step="1" name="table_rate"
required placeholder="50.00"><br><br>

            <h3>Members</h3>
            <label>Member 1 Name:</label><br>
            <input type="text" name="table_members_1" required
placeholder="e.g. Richard Hammond"><br><br>

            <label>Member 2 Name:</label><br>
            <input type="text" name="table_members_2" required
placeholder="e.g. James May"><br>
        </fieldset>

        <button type="submit" style="padding: 10px 20px;
background-color: #28a745; color: white; border: none; cursor:
pointer; font-size: 16px;">
            Add Teams to Database
        </button>
    </form>
{% endblock %}

```

base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}TB Promotions{% endblock %}</title>

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com"
crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Zilla+Slab:wght@700&display=swap" rel="stylesheet">

  <style>
    body {
      font-family: Arial;
      margin: 0;
      padding: 0;
      background-color: #f5f5f5;
    }

    .site-header {
      background-color: #222;
      color: white;
      padding: 30px 20px;
      text-align: center;
    }

    .site-header h1 {
```

```
margin: 0;
font-family: 'Zilla Slab';
font-size: 6rem;
}
```

```
nav {
  background-color: #333;
  display: flex;
  align-items: center;
  padding: 15px 25px;
}
```

```
.nav-logo img {
  height: 100px;
  width: auto;
  margin-right: 25px;
}
```

```
.nav-links {
  display: flex;
  gap: 25px;
  flex-wrap: wrap;

  border-left: 3px solid #29516c;
  padding: 25px 0 25px 20px;
  margin-left: 5px;
}
```

```
nav a {
```

```
    color: white;
    text-decoration: none;
    font-weight: bold;
    display: inline-block
}
```

```
nav a:hover {
    text-decoration: underline;
    transform: scale(1.1);
    color: #ffffff;
}
```

```
main {
    padding: 20px;
    max-width: 900px;
    margin: 0 auto 60px auto;
}
```

```
footer {
    background-color: #333;
    color: white;
    text-align: center;
    padding: 10px;
    position: fixed;
    bottom: 0;
    width: 100%;
    font-family: 'Zilla Slab';
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<header class="site-header">
```

```
    <h1>TB Promotions</h1>
```

```
</header>
```

```
<nav>
```

```
    <a href="/" class="nav-logo">
```

```
        
```

```
    </a>
```

```
    <div class="nav-links">
```

```
        <a href="/suitable_team_for_venue">Suitable team for
venue</a>
```

```
        <a href="/suitable_team_for_target_audience">Suitable team
for target audience</a>
```

```
        <a href="/teams_with_one_solo_member">Teams with one solo
member</a>
```

```
        <a href="/venue_with_3_or_more_target_audiences">Venue
with 3 or more target audiences</a>
```

```
        <a href="/non_solo_teams">Non solo teams</a>
```

```
        <a href="/booking_costs_from_team">Booking costs(team)</a>
```

```
        <a href="/booking_costs_from_venue">Booking
costs(venue)</a>
```

```
        <a href="/add_teams">Add new teams</a>
```

```
    </div>
```

```
</nav>
```

```
<main>
```

```
    {% block content %}
```

```
    {% endblock %}
```

```
</main>
```

```
<footer>
    TB Promotions Tracker (TBPT)
</footer>

</body>
</html>
```

booking_costs_from_team.html

```
{% extends "base.html" %}

{% block title %}Event Cost & Budget Analysis Through Team{%
endblock %}

{% block content %}
    <h1>Event Cost & Budget Analysis Through Team</h1>

    <form method="POST" action="/booking_costs_from_team">
        <label><strong>Choose a Team:</strong></label>
        <select name="team_selection_id" required>
            <option value="" disabled selected>Select a
team...</option>
            {% for team in teams_dropdown %}
                <option value="{{ team.team_id }}">{{ team.name }}
</option>
            {% endfor %}
        </select>
        <button type="submit">Search Matches</button>
    </form>

    <hr>
```

```

{% if selected_name %}
    <h2>Results for: {{ selected_name }}</h2>

    <table border="1" style="width: 100%; border-collapse:
collapse; margin-top: 20px;">
        <thead style="background-color: #f2f2f2;">
            <tr>
                <th>Venue</th>
                <th>Start Time</th>
                <th>End Time</th>
                <th>Hourly Rate</th>
                <th>Hours</th>
                <th>Total Cost</th>
                <th>Max Fee</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody>
            {% for row in bookings %}
            <tr style = "font-weight: bold;
                text-align: center;
                {% if row.booking_status == 'OVER BUDGET'
%}
                    background-color: #ffdce0; color:
#af0000;
                {% elif row.booking_status == 'WITHIN
BUDGET' %}
                    background-color: #d4edda; color:
#155724;
                {% endif %}
            ">
            <td>{{ row.venue_name }}</td>

```

```

        <td>{{ row.start_time }}</td>
        <td>{{ row.end_time }}</td>
        <td>£{{ row.team_hourly_rate }}</td>
        <td>{{ row.duration_in_hours }}</td>
        <td><strong>£{{ row.total_booking_cost
    }}</strong></td>

        <td>£{{ row.venue_max_fee }}</td>
        <td>{{ row.booking_status }}</td>
    </tr>
    {% endfor %}
</tbody>
</table>
{% endif %}

{% endblock %}

```

booking_costs_from_venue.html

```

{% extends "base.html" %}

{% block title %}Event Cost & Budget Analysis Through Venue{%
endblock %}

{% block content %}
    <h1>Event Cost & Budget Analysis Through Venue</h1>

    <form method="POST" action="/booking_costs_from_venue">
        <label><strong>Select a Venue:</strong></label>
        <select name="venue_selection_id" required>

```

```
<option value="" disabled selected>Choose a
location...</option>
    {% for venue in venues_dropdown %}
        <option value="{{ venue.venue_id }}">{{ venue.name
    }}</option>
    {% endfor %}
</select>
<button type="submit">Show Bookings</button>
</form>

<hr>

{% if selected_name %}
    <h2>Bookings at: {{ selected_name }}</h2>

    {% if bookings|length == 0 %}
        <p>No bookings found for this venue.</p>
    {% else %}
        <table border="1" style="width: 100%; border-collapse:
collapse; margin-top: 20px;">
            <thead style="background-color: #e0e0e0;">
                <tr>
                    <th>Team Name</th>
                    <th>Start Date/Time</th>
                    <th>End Date/Time</th>
                    <th>Duration (Hrs)</th>
                    <th>Team Rate</th>
                    <th>Total Cost</th>
                    <th>Venue Max Limit</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
```

```

<tbody>
  {% for row in bookings %}
    <tr style="font-weight: bold;
      text-align: center;
      {% if row.booking_status == 'OVER
BUDGET' %}
        background-color: #ffdce0;
color: #af0000;
      {% elif row.booking_status ==
'WITHIN BUDGET' %}
        background-color: #d4edda;
color: #155724;
      {% endif %}
    ">
    <td>{{ row.team_name }}</td>
    <td>{{ row.start_time }}</td>
    <td>{{ row.end_time }}</td>
    <td>{{ row.duration_in_hours }}</td>
    <td>£{{ row.team_hourly_rate }}</td>
    <td>£{{ row.total_booking_cost }}</td>
    <td>£{{ row.venue_max_fee }}</td>
    <td>{{ row.booking_status }}</td>
  </tr>
  {% endfor %}
</tbody>
</table>
{% endif %}
{% endif %}

{% endblock %}

```

home.html

```
{% extends "base.html" %}

{% block title %}Home{% endblock %}

{% block content %}
    <h1>TeamBuilder Promotions Booking System</h1>
    <p>Please use the navigation bar.</p>
{% endblock %}
```

non_solo_teams.html

```
{% extends "base.html" %}

{% block title %}Members of Non-Solo Teams{% endblock %}

{% block content %}
    <h1>Members of Non-Solo Teams</h1>

    {% if team_members %}
        <table border="1">
            <thead>
                <tr>
                    <th>Team Name</th>
                    <th>Member Name</th>
                </tr>
            </thead>
```

```
<tbody>
    {% for row in team_members %}
    <tr>
        <td>{{ row.team_name }}</td>
        <td>{{ row.member_name }}</td>
    </tr>
    {% endfor %}
</tbody>
</table>
{% else %}
    <p>No teams with multiple members were found.</p>
{% endif %}
{% endblock %}
```

suitable_team_for_target_audience.html

```
{% extends "base.html" %}

{% block title %}Find Suitable Teams for a Target Audience{%
endblock %}

{% block content %}
    <h1>Find Suitable Teams for a Target Audience</h1>

    <form method="POST"
action="/suitable_team_for_target_audience">
        <label for="target_audience_id"><strong>Choose an
Audience:</strong></label>
```

```

    <select name="target_audience_id" id="target_audience_id"
required>
        <option value="" disabled selected>Select an
audience...</option>
        {% for audience in audiences %}
            <option
value="{{ audience.target_audience_id }}">{{ audience.category }}
</option>
        {% endfor %}
    </select>
    <button type="submit">Search Matches</button>
</form>

```

```

{% if selected_venue %}
    <h2>Results for Audience: {{ selected_venue }}</h2>
    {% if teams %}
        <table border="1">
            <thead>
                <tr>
                    <th>Team Name</th>
                    <th>Matched Audience</th>
                    <th>Hourly Rate</th>
                    <th>Suitable Venue</th>
                </tr>
            </thead>
            <tbody>
                {% for team in teams %}
                    <tr>
                        <td>{{ team.team_name }}</td>
                        <td>{{ team.category }}</td>
                        <td>£{{ team.hourly_rate }}</td>
                        <td>{{ team.venue_name }}</td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    {% else %}
        <p>No teams found for this audience.</p>
    {% endif %}
{% endif %}

```

```

        </tr>
        {% endfor %}
    </tbody>
</table>
{% else %}
    <p>No teams found for this audience type.</p>
{% endif %}
{% endif %}
{% endblock %}

```

suitable_team_for_venue.html

```

{% extends "base.html" %}

{% block title %}Find Suitable Teams for a Venue{% endblock %}

{% block content %}
    <h1>Find Suitable Teams for a Venue</h1>

    <form method="POST" action="/suitable_team_for_venue">
        <label for="venue_id"><strong>Choose a
Venue:</strong></label>
        <select name="venue_id" id="venue_id" required>
            <option value="" disabled selected>Select a
venue...</option>
            {% for venue in venues %}
                <option value="{{ venue.venue_id }}">{{ venue.name
}}</option>
            {% endfor %}
    </form>

```

```

</select>
<button type="submit">Search Matches</button>
</form>

{% if selected_venue %}
  <h2>Results for: {{ selected_venue }}</h2>
  {% if teams %}
    <table border="1">
      <thead>
        <tr>
          <th>Team Name</th>
          <th>Matched Audience</th>
          <th>Hourly Rate</th>
        </tr>
      </thead>
      <tbody>
        {% for team in teams %}
          <tr>
            <td>{{ team.name }}</td>
            <td>{{ team.category }}</td>
            <td>£{{ team.hourly_rate }}</td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
  {% else %}
    <p>No teams found for this venue.</p>
  {% endif %}
{% endif %}
{% endblock %}

```

teams_with_one_solo_member.html

```
{% extends "base.html" %}

{% block title %}Solo Member Teams{% endblock %}

{% block content %}
    <h1>Solo Member Teams</h1>

    {% if teams %}
        <table border="1">
            <thead>
                <tr>
                    <th>Team Name</th>
                    <th>Solo Member Name</th>
                </tr>
            </thead>
            <tbody>
                {% for team in teams %}
                    <tr>
                        <td>{{ team.team_name }}</td>
                        <td>{{ team.member_name }}</td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    {% else %}
        No teams found.
    {% endif %}
{% endblock %}
```

```
        {% endfor %}
    </tbody>
</table>
{% else %}
    <p>No solo teams found in the database.</p>
{% endif %}
{% endblock %}
```

venue_with_3_or_more_target_audiences.html

```
{% extends "base.html" %}

{% block title %}Venues Hosting 3+ Target Audiences{% endblock %}

{% block content %}
    <h1>Venues Hosting 3+ Target Audiences</h1>

    {% if venues %}
        <table border="1">
            <thead>
                <tr>
                    <th>Venue Name</th>
                    <th>Count</th>
                    <th>Audience Categories</th>
                </tr>
            </thead>
            <tbody>
                {% for venue in venues %}
                    <tr>
```

```
        <td>{{ venue.venue_name }}</td>
        <td>{{ venue.audience_count }}</td>
        <td>{{ venue.audience_list }}</td>
    </tr>
    {% endfor %}
</tbody>
</table>
{% else %}
    <p>No venues found hosting 3 or more target audiences.</p>
{% endif %}
{% endblock %}
```

assessment_1_database.sql

```
DROP DATABASE IF EXISTS `booking_system`;
```

```
CREATE DATABASE `booking_system` DEFAULT CHARACTER SET utf8mb4;
```

```
USE `booking_system`;
```

```
--
```

```
-- Table: target_audience
```

```
--
```

```
DROP TABLE IF EXISTS `target_audience`;
```

```
CREATE TABLE `target_audience` (  
  `target_audience_id` INT NOT NULL AUTO_INCREMENT,  
  `category` VARCHAR(50) NOT NULL DEFAULT '',  
  CONSTRAINT `pk_target_audience` PRIMARY KEY  
  (`target_audience_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--
```

```
-- Table: venue
```

```
--
```

```
DROP TABLE IF EXISTS `venue`;
```

```
CREATE TABLE `venue` (  
  `venue_id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(50) NOT NULL DEFAULT '',  
  `capacity` INT NOT NULL DEFAULT 0,  
  `max_booking_fee` DECIMAL(10,2) NOT NULL DEFAULT 0.00,
```

```
`location` VARCHAR(255) NOT NULL DEFAULT '',  
`phone` VARCHAR(20) NOT NULL DEFAULT '',  
`email` VARCHAR(255) NOT NULL DEFAULT '',  
CONSTRAINT `pk_venue` PRIMARY KEY (`venue_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--  
-- Table: expert_guide  
--
```

```
DROP TABLE IF EXISTS `expert_guide`;
```

```
CREATE TABLE `expert_guide` (  
  `expert_guide_id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(50) NOT NULL DEFAULT '',  
  `phone` VARCHAR(20) NOT NULL DEFAULT '',  
  `email` VARCHAR(255) NOT NULL DEFAULT '',  
  CONSTRAINT `pk_expert_guide` PRIMARY KEY (`expert_guide_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--  
-- Table: team  
--
```

```
DROP TABLE IF EXISTS `team`;
```

```
CREATE TABLE `team` (  
  `team_id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(50) NOT NULL DEFAULT '',  
  `hourly_rate` DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
  CONSTRAINT `pk_team` PRIMARY KEY (`team_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--  
-- Table: member  
--
```

```
DROP TABLE IF EXISTS `member`;
```

```
CREATE TABLE `member` (  
  `member_id` int NOT NULL AUTO_INCREMENT,  
  `name` varchar(50) NOT NULL DEFAULT '',  
  `phone` varchar(20) NOT NULL DEFAULT '',  
  `email` varchar(255) NOT NULL DEFAULT '',
```

```
CONSTRAINT `pk_member` PRIMARY KEY (`member_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--  
-- Table: presentation  
--
```

```
DROP TABLE IF EXISTS `presentation`;
```

```
CREATE TABLE `presentation` (  
  `presentation_id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(50) NOT NULL DEFAULT '',  
  `length_in_hours` DECIMAL(4,2) NOT NULL DEFAULT 0.00,  
  `topic` VARCHAR(100) NOT NULL DEFAULT '',  
  `file_path` VARCHAR(255) NOT NULL DEFAULT '',  
  CONSTRAINT `pk_presentation` PRIMARY KEY (`presentation_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--  
-- Table: team_has_member (Join Table)  
--
```

```
DROP TABLE IF EXISTS `team_has_member`;
```

```
CREATE TABLE `team_has_member` (  
  `team_has_member_id` INT NOT NULL AUTO_INCREMENT,  
  `team_id` INT NOT NULL,  
  `member_id` INT NOT NULL,  
  CONSTRAINT `pk_team_has_member` PRIMARY KEY  
  (`team_has_member_id`),  
  CONSTRAINT `fk_team_has_member_to_team` FOREIGN KEY (`team_id`)  
  REFERENCES `team` (`team_id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_team_has_member_to_member` FOREIGN KEY  
  (`member_id`) REFERENCES `member` (`member_id`) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--
```

```
-- Table: team_presentation (Join Table)
```

```
--
```

```
DROP TABLE IF EXISTS `team_presentation`;
```

```
CREATE TABLE `team_presentation` (  
  `team_presentation_id` INT NOT NULL AUTO_INCREMENT,  
  `team_id` INT NULL, # allowing empty because not all teams must  
  have a booking  
  `presentation_id` INT NULL, # allowing empty because not all  
  presentations must have a booking  
  CONSTRAINT `pk_team_presentation` PRIMARY KEY  
  (`team_presentation_id`),
```

```
CONSTRAINT `fk_team_presentation_to_team` FOREIGN KEY
(`team_id`) REFERENCES `team` (`team_id`) ON DELETE CASCADE,
CONSTRAINT `fk_team_presentation_to_presentation` FOREIGN KEY
(`presentation_id`) REFERENCES `presentation` (`presentation_id`)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--
-- Table: booking
--
```

```
DROP TABLE IF EXISTS `booking`;
```

```
CREATE TABLE booking (
  `booking_id` INT NOT NULL AUTO_INCREMENT,
  `team_presentation_id` INT NOT NULL, # this is not null on
purpose
  `venue_id` INT NULL, # allowing because not all venues must
have bookings
  `start_datetime` DATETIME NOT NULL DEFAULT '1000-01-01
00:00:00',
  `end_datetime` DATETIME NOT NULL DEFAULT '1000-01-01 00:00:00',
  CONSTRAINT `pk_booking` PRIMARY KEY (`booking_id`),
  CONSTRAINT `fk_booking_to_team_presentation_id` FOREIGN KEY
(`team_presentation_id`) REFERENCES `team_presentation`
(`team_presentation_id`) ON DELETE CASCADE,
  CONSTRAINT `fk_booking_to_venue` FOREIGN KEY (`venue_id`)
REFERENCES `venue` (`venue_id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

--

-- Table: team_audience (Join Table)

--

DROP TABLE IF EXISTS `team\_audience`;

```
CREATE TABLE `team_audience` (  
  `team_audience_id` INT NOT NULL AUTO_INCREMENT,  
  `team_id` INT NOT NULL,  
  `target_audience_id` INT NOT NULL,  
  CONSTRAINT `pk_team_audience` PRIMARY KEY (`team_audience_id`),  
  CONSTRAINT `fk_team_audience_to_team` FOREIGN KEY (`team_id`)  
REFERENCES `team` (`team_id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_team_audience_to_target_audience` FOREIGN KEY  
(`target_audience_id`) REFERENCES `target_audience`  
(`target_audience_id`) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

--

-- Table: presentation_audience (Join Table)

--

```
DROP TABLE IF EXISTS `presentation_audience`;
```

```
CREATE TABLE `presentation_audience` (  
  `presentation_audience_id` INT NOT NULL AUTO_INCREMENT,  
  `presentation_id` INT NOT NULL,  
  `target_audience_id` INT NOT NULL,  
  CONSTRAINT `pk_presentation_audience` PRIMARY KEY  
  (`presentation_audience_id`),  
  CONSTRAINT `fk_presentation_audience_to_presentation` FOREIGN  
  KEY (`presentation_id`) REFERENCES `presentation`  
  (`presentation_id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_presentation_audience_to_target_audience` FOREIGN  
  KEY (`target_audience_id`) REFERENCES `target_audience`  
  (`target_audience_id`) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--
```

```
-- Table: expert_guide_audience (Join Table)
```

```
--
```

```
DROP TABLE IF EXISTS `expert_guide_audience`;
```

```
CREATE TABLE `expert_guide_audience` (  
  `expert_guide_audience_id` INT NOT NULL AUTO_INCREMENT,
```

```
`expert_guide_id` INT NOT NULL,  
`target_audience_id` INT NOT NULL,  
CONSTRAINT `pk_expert_guide_audience` PRIMARY KEY  
(`expert_guide_audience_id`),  
CONSTRAINT `fk_expert_guide_audience_guide` FOREIGN KEY  
(`expert_guide_id`) REFERENCES `expert_guide` (`expert_guide_id`)  
ON DELETE CASCADE,  
CONSTRAINT `fk_expert_guide_audience_target` FOREIGN KEY  
(`target_audience_id`) REFERENCES `target_audience`  
(`target_audience_id`) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
--  
-- Table: venue_audience (Join Table)  
--
```

```
DROP TABLE IF EXISTS `venue_audience`;
```

```
CREATE TABLE `venue_audience` (  
  `venue_audience_id` INT NOT NULL AUTO_INCREMENT,  
  `venue_id` INT NOT NULL,  
  `target_audience_id` INT NOT NULL,  
  CONSTRAINT `pk_venue_audience` PRIMARY KEY  
(`venue_audience_id`),
```

```
CONSTRAINT `fk_venue_audience_venue` FOREIGN KEY (`venue_id`)
REFERENCES `venue` (`venue_id`) ON DELETE CASCADE,

CONSTRAINT `fk_venue_audience_target` FOREIGN KEY
(`target_audience_id`) REFERENCES `target_audience`
(`target_audience_id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

assesment_1_database_adding_data.sql

```
USE `booking_system`;
```

```
--
```

```
-- Dumping data for table `target_audience`
```

```
--
```

```
SET autocommit=0;
```

```
INSERT INTO `target_audience` (`target_audience_id`, `category`)
VALUES
```

```
(1, 'General Public'),
```

```
(2, 'Freelancers'),
```

```
(3, 'UK Government'),
```

```
(4, 'University Students'),
```

```
(5, 'University Graduates'),
```

```
(6, 'Small Businesses'),
```

```
(7, 'Medium Businesses'),
```

```
(8, 'Policy Makers');
```

```
COMMIT;
```

--

-- Dumping data for table `venue`

--

```
SET autocommit=0;
```

```
INSERT INTO `venue` (`venue_id`, `name`, `capacity`,  
`max_booking_fee`, `location`, `phone`, `email`) VALUES
```

```
(1, 'UWE Exhibition & Conference Centre', 500, 1200.00, 'Frenchay  
Campus, Bristol BS16 1QY', '0117 1023495', 'events@uwe.ac.uk'),
```

```
(2, 'University of Bristol - Wills Memorial Building', 300,  
1500.00, 'Queens Rd, Bristol BS8 1RJ', '0227 1023495',  
'bookings@bristol.su'),
```

```
(3, 'Engine Shed', 60, 450.00, 'Station Approach, Temple Meads,  
Bristol BS1 6QH', '0227 102948', 'venue.hire@engine-shed.co.uk'),
```

```
(4, 'Easton Community Centre', 100, 200.00, 'Kilburn St, Easton,  
Bristol BS5 6AW', '0333 123400',  
'bookings@eastoncommunitycentre.su'),
```

```
(5, 'Bristol Marriott Royal Hotel', 400, 2500.00, 'College Green,  
Bristol BS1 5TA', '07434 607192', 'events@marriottbristol.co.uk'),
```

```
(6, 'Future Leap Clifton', 40, 350.00, '1-3 Gloucester Rd, Bristol  
BS7 8AA', '07434 958273', 'venue.hire@futureleap.co.uk'),
```

```
(7, 'The Bristol Hotel', 200, 900.00, 'Prince St, Bristol BS1  
4QF', '07434 1032483', 'venue.hire@thebristolhotel.com'),
```

```
(8, 'Paintworks Event Space', 250, 1100.00, 'Bath Rd, Bristol BS4  
3EH', '07434 1029358', 'venue.hire@paintworks.su'),
```

```
(9, 'We The Curious - Rosary Room', 150, 800.00, 'One Millennium  
Square, Bristol BS1 5DB', '07434 2046050',  
'venue.hire@wethecurious.co.uk'),
```

```
(10, 'The Watershed', 80, 550.00, '1 Canons Rd, Bristol BS1 5TX',  
'07434 92835100', 'venue.hire@watershed.co.uk');
```

```
COMMIT;
```

```
--  
-- Dumping data for table `venue_audience`  
--  
  
SET autocommit=0;  
INSERT INTO `venue_audience` (`venue_id`, `target_audience_id`)  
VALUES  
(1, 4),  
(1, 5),  
(1, 6),  
(2, 4),  
(2, 5),  
(2, 2),  
(2, 6),  
(3, 2),  
(3, 6),  
(4, 1),  
(5, 6),  
(5, 4),  
(5, 1),  
(5, 2),  
(6, 3),  
(7, 5),  
(8, 2),  
(9, 1),  
(10, 2);  
COMMIT;
```

```
--  
-- Dumping data for table `expert_guide`  
--  
  
SET autocommit=0;  
INSERT INTO `expert_guide` (`expert_guide_id`, `name`, `phone`,  
`email`) VALUES  
(1, 'Paul Jackson', '4411732 81912', 'Paul4.Jackson@uwe.ac.uk'),  
(2, 'James Barrett', '07700 900202', 'James6.Barrett@uwe.ac.uk'),  
(3, 'Angus Hutchison', '07700 900203',  
'Angus.Hutchison@uwe.ac.uk'),  
(4, 'Andrew McCarthy', '07700 900204',  
'Andrew6.Mccarthy@uwe.ac.uk'),  
(5, 'Tunde Oduguwa', '07700 900205', 'Tunde.Oduguwa@uwe.ac.uk'),  
(6, 'Elizabeth White', '4411 73287245',  
'Elizabeth.White@uwe.ac.uk'),  
(7, 'Peter Case', '4411 73281709', 'Peter.Case@uwe.ac.uk'),  
(8, 'Susanna Clasby', '4411 73287245',  
'Susanna.Clasby@uwe.ac.uk'),  
(9, 'Jim Campbell', '4411 73285833', 'Jim.Campbell@uwe.ac.uk'),  
(10, 'Jim workbell', '0192 10293857', 'Jim.workbell@uwe.ac.uk'),  
(11, 'Ian Johnson', '4411 01923894', 'Ian.Johnson@uwe.ac.uk');  
COMMIT;  
  
--  
-- Dumping data for table `team`  
--
```

```
SET autocommit=0;
INSERT INTO `team` (`team_id`, `name`, `hourly_rate`) VALUES
(1, 'Alpha Team', 180.00),
(2, 'Bravo Team', 165.00),
(3, 'Charlie Team', 140.00),
(4, 'Delta Team', 250.00),
(5, 'Echo Team', 250.00),
(6, 'Foxtrot Team', 130.00),
(7, 'Golf Team', 195.00),
(8, 'Hotel Team', 175.00),
(9, 'India Team', 210.00),
(10, 'Juliet Team', 155.00);
COMMIT;
```

--

-- Dumping data for table `member`

--

```
SET autocommit=0;
INSERT INTO `member` (`member_id`, `name`, `phone`, `email`)
VALUES
(1, 'George Atreides', '07700 900548',
'george.a@tbpromotions.co.uk'),
(2, 'Zahra Mohammed', '07700 900102',
'zahra.m@tbpromotions.co.uk'),
(3, 'Toby Henderson', '07700 406203',
'toby.h@tbpromotions.co.uk'),
```

(4, 'Ollie Thompson', '07700 193847',
'ollie.t@tbpromotions.co.uk'),
(5, 'Muna Abdi', '07700 1092958', 'muna.a@tbpromotions.co.uk'),
(6, 'Aidan Gallagher', '07773 203440',
'aidan.g@tbpromotions.co.uk'),
(7, 'Alisha Khan', '07773 193205', 'alisha.k@tbpromotions.co.uk'),
(8, 'Hala Ibrahim', '07773 957174', 'hala.i@tbpromotions.co.uk'),
(9, 'Rizwan Ahmed', '07773 295817',
'rizwan.a@tbpromotions.co.uk'),
(10, 'Sagal Farah', '07773 038381', 'sagal.f@tbpromotions.co.uk'),
(11, 'Sam Edwards', '07733 819248', 'sam.e@tbpromotions.co.uk'),
(12, 'Zainab Ali', '07733 900122', 'zainab.a@tbpromotions.co.uk'),
(13, 'Zakarie Duale', '07733 900102',
'zakarie.d@tbpromotions.co.uk'),
(14, 'Kajally Jobe', '07733 900123',
'kajally.j@tbpromotions.co.uk'),
(15, 'Zak Barker', '07733 900124', 'zak.b@tbpromotions.co.uk'),
(16, 'Anish Adhikari', '07700 900129',
'anish.a@tbpromotions.co.uk'),
(17, 'Ayub Aden', '07700 800293', 'ayub.a@tbpromotions.co.uk'),
(18, 'Shovit Adhikari', '07700 700394',
'shovit.a@tbpromotions.co.uk'),
(19, 'Sujal Adhikari', '07700 918239',
'sujal.a@tbpromotions.co.uk'),
(20, 'Leila Hassan', '07700 900881',
'leila.h@tbpromotions.co.uk'),
(21, 'Marcus Thorne', '07700 900772',
'marcus.t@tbpromotions.co.uk'),
(22, 'Chloe Whittaker', '07773 405912',
'chloe.w@tbpromotions.co.uk'),
(23, 'Arjun Singh', '07733 112233', 'arjun.s@tbpromotions.co.uk'),
(24, 'Fiona Gallagher', '07700 304050',
'fiona.g@tbpromotions.co.uk'),
(25, 'Omar Sharif', '07773 667788', 'omar.s@tbpromotions.co.uk'),
(26, 'Elena Rossi', '07733 998877', 'elena.r@tbpromotions.co.uk'),

```
(27, 'David Okoro', '07700 554433', 'david.o@tbpromotions.co.uk'),
(28, 'Sophie Bennett', '07773 121212',
'sophie.b@tbpromotions.co.uk'),
(29, 'Liam O'Connor', '07733 343434',
'liam.o@tbpromotions.co.uk'),
(30, 'Maya Patel', '07700 789456', 'maya.p@tbpromotions.co.uk'),
(31, 'Noah Williams', '07773 456789',
'noah.w@tbpromotions.co.uk'),
(32, 'Isabella Martinez', '07733 654321',
'isabella.m@tbpromotions.co.uk'),
(33, 'Yusuf Mansour', '07700 111222',
'yusuf.m@tbpromotions.co.uk'),
(34, 'Grace Taylor', '07773 333444',
'grace.t@tbpromotions.co.uk');
COMMIT;
```

```
--
```

```
-- Dumping data for table `team_has_member`
```

```
--
```

```
SET autocommit=0;
```

```
INSERT INTO `team_has_member` (`team_id`, `member_id`) VALUES
```

```
(2,3),
```

```
(2,4),
```

```
(3,5),
```

```
(3,6),
```

```
(4,7),
```

```
(4,8),
```

```
(5,9),
```

(5, 10),
(7, 13),
(8, 14),
(9, 15),
(10, 1),
(1, 3),
(2, 5),
(3, 7),
(4, 9),
(5, 16),
(8, 18),
(8, 19),
(2, 22),
(2, 23),
(3, 24),
(3, 25),
(4, 26),
(4, 27),
(5, 28),
(5, 29),
(6, 30),
(7, 31),
(8, 32),
(9, 33),
(2, 28),
(3, 30),
(4, 32),
(5, 34),
(8, 23),
(8, 24);
COMMIT;

```
--  
-- Dumping data for table `presentation`  
--  
  
SET autocommit=0;  
INSERT INTO `presentation` (`presentation_id`, `name`,  
`length_in_hours`, `topic`, `file_path`) VALUES  
(1, 'How To Beat Your Competition', 3, 'Business Strategy',  
'/home/TeamBuilder-Promotions/database-files/Beat Your  
Competition.pptx'),  
(2, 'How To Monopolise Your Industry', 2, 'Business Strategy',  
'/home/TeamBuilder-Promotions/database-files/Monopolise Your  
Industry 101.pptx'),  
(3, 'How To Be More Motivated', 2, 'Motivational',  
'/home/TeamBuilder-Promotions/database-files/Motivate-The-  
Motivation.pptx'),  
(4, 'Sustainability!', 1, 'Environmental', '/home/TeamBuilder-  
Promotions/database-files/Sustainability-Is-Important-v13.pptx'),  
(5, 'An Exciting Strategic Business Presentation', 1,  
'Technology',  
'/home/TeamBuilder-Promotions/database-files/Exciting Strategic  
Business Presentation-FINAL.pptx'),  
(6, 'How To Be A Team Player!', 2, 'Motivational',  
'/home/TeamBuilder-Promotions/database-files/Team Player  
Presentation.pptx'),  
(7, 'Guide On Overworking Your University Graduates ', 2,  
'Business Strategy', '/home/TeamBuilder-Promotions/database-  
files/Guide On Overworking Employees.pptx');  
COMMIT;
```

```
--  
-- Dumping data for table `team_presentation`  
--
```

```
SET autocommit=0;  
INSERT INTO `team_presentation` (`team_id`, `presentation_id`)  
VALUES  
(1,1),  
(4,1),  
(2,2),  
(10,3),  
(3,4),  
(5,5),  
(7,1),  
(9,5),  
(5, 2),  
(1, 2),  
(2, 1),  
(3, 6),  
(4, 2),  
(6, 4),  
(7, 7),  
(8, 3),  
(9, 6),  
(10, 6),  
(1, 7),  
(5, 1);  
COMMIT;
```

```
--  
-- Dumping data for table `booking`  
--  
  
SET autocommit=0;  
INSERT INTO `booking` (`team_presentation_id`, `venue_id`,  
`start_datetime`, `end_datetime`) VALUES  
(1, 1, '2026-03-10 09:00:00', '2026-03-10 12:00:00'),  
(3, 3, '2026-03-15 13:00:00', '2026-03-15 15:00:00'),  
(5, 4, '2026-03-22 10:00:00', '2026-03-22 11:00:00'),  
(6, 5, '2026-04-05 09:30:00', '2026-04-05 10:30:00'),  
(4, 2, '2026-04-12 14:00:00', '2026-04-12 16:00:00'),  
(2, 2, '2026-04-20 10:00:00', '2026-04-20 13:00:00'),  
(5, 6, '2026-06-15 11:00:00', '2026-06-15 12:00:00'),  
(3, 10, '2026-07-01 10:00:00', '2026-07-01 12:00:00'),  
(7, 7, '2026-05-14 10:00:00', '2026-05-14 13:00:00'),  
(1, 8, '2026-06-02 14:00:00', '2026-06-02 17:00:00'),  
(9, 4, '2026-08-10 10:00:00', '2026-08-10 12:00:00'),  
(1, 2, '2026-09-05 10:00:00', '2026-09-05 13:00:00'),  
(2, 5, '2026-09-12 14:00:00', '2026-09-12 16:00:00'),  
(3, 1, '2026-09-20 09:00:00', '2026-09-20 11:00:00'),  
(4, 9, '2026-10-02 11:00:00', '2026-10-02 13:00:00'),  
(5, 3, '2026-10-15 15:00:00', '2026-10-15 16:00:00'),  
(6, 8, '2026-11-01 10:00:00', '2026-11-01 11:00:00'),  
(7, 1, '2026-11-10 13:00:00', '2026-11-10 16:00:00'),  
(8, 6, '2026-11-20 09:00:00', '2026-11-20 10:00:00'),  
(9, 10, '2026-12-05 14:00:00', '2026-12-05 15:00:00'),  
(2, 7, '2026-12-12 10:00:00', '2026-12-12 12:00:00');  
COMMIT;
```

--

-- Dumping data for table `team\_audience`

--

SET autocommit=0;

INSERT INTO `team\_audience` (`team\_id`, `target\_audience\_id`)
VALUES

(1, 4),

(1, 5),

(2, 6),

(3, 1),

(4, 4),

(5, 6),

(6, 3),

(7, 5),

(8, 2),

(9, 2),

(10, 4);

COMMIT;

--

-- Dumping data for table `presentation\_audience`

--

```
SET autocommit=0;
INSERT INTO `presentation_audience` (`presentation_id`,
`target_audience_id`) VALUES
(1, 4),
(1, 6),
(2, 6),
(3, 4),
(3, 5),
(4, 1),
(5, 6),
(5, 3),
(6, 1),
(6, 4),
(7, 6),
(7, 7);
COMMIT;
```

--

-- Dumping data for table `expert\_guide\_audience`

--

```
SET autocommit=0;
INSERT INTO `expert_guide_audience` (`expert_guide_id`,
`target_audience_id`) VALUES
(1, 5),
(2, 2),
(3, 3),
(4, 4),
(5, 1),
```

```
(1, 2),  
(6, 4),  
(7, 8),  
(8, 3),  
(9, 7),  
(10, 8),  
(11, 2),  
(11, 3),  
(3, 2),  
(6, 5),  
(2, 1),  
(1,4);  
COMMIT;
```

Example commands slightly edited to work in MYSQL workbench and shown in video

part_1.sql

#part 1

#A list of teams whose target audiences are suitable to perform at a chosen venue (return at least 2 teams). Choose the target audiences and a suitable venue.

```
SELECT DISTINCT  
    team.name,  
    team.hourly_rate,  
    target_audience.category  
FROM team
```

```
JOIN team_audience
      ON team.team_id = team_audience.team_id
JOIN target_audience
      ON team_audience.target_audience_id =
target_audience.target_audience_id
JOIN venue_audience
      ON target_audience.target_audience_id =
venue_audience.target_audience_id
WHERE venue_audience.venue_id = 1
```

part_2.sql

#part 2

#A list of teams whose target audiences are suitable to perform at a chosen venue (return at least 2 teams). Choose the target audiences and a suitable venue.

```
SELECT DISTINCT
      team.name AS team_name,
      team.hourly_rate,
      target_audience.category,
      venue.name AS venue_name
FROM team
JOIN team_audience
      ON team.team_id = team_audience.team_id
JOIN target_audience
      ON team_audience.target_audience_id =
target_audience.target_audience_id
JOIN venue_audience
      ON target_audience.target_audience_id =
venue_audience.target_audience_id
```

```
        JOIN venue
            ON venue_audience.venue_id = venue.venue_ID
WHERE target_audience.target_audience_id = 1;
```

part_3.sql

#part 3

Output at least 2 teams that have just one solo member (must include member's name) in the database.

```
SELECT
    team.name AS team_name,
    MAX(member.name) AS member_name
FROM team
JOIN team_has_member
    ON team.team_id = team_has_member.team_id
JOIN member
    ON member.member_id = team_has_member.member_id
GROUP BY team.team_id, team.name
HAVING COUNT(team_has_member.member_id) = 1;
```

part_4.sql

#part 4

#A list of venues that host teams from 3 or more target audiences (return at least 2 venues)

```
SELECT
    venue.name AS venue_name,
    COUNT(DISTINCT target_audience.target_audience_id) AS
audience_count,
```

```
        GROUP_CONCAT(DISTINCT target_audience.category
SEPARATOR ', ') AS audience_list
FROM venue
JOIN venue_audience ON venue.venue_id =
venue_audience.venue_id
JOIN target_audience ON venue_audience.target_audience_id
= target_audience.target_audience_id
GROUP BY venue.venue_id, venue.name
HAVING audience_count >= 3;
```

part_5.sql

#part 5

#create a query to show all the members of 2 non-solo teams,
showing clearly which team each member belongs to

```
SELECT
    team.name AS team_name,
    member.name AS member_name
FROM team
JOIN team_has_member ON team.team_id =
team_has_member.team_id
JOIN member ON team_has_member.member_id =
member.member_id
WHERE team.team_id IN (
    SELECT team_id
    FROM team_has_member
    GROUP BY team_id
    HAVING COUNT(member_id) > 1
)
ORDER BY team.name;
```

part_6.sql

#part 6

#A booking has a start time and end time, and each team has a fee in £ per hour. Write a query in Python to

#run a calculated field of the total event cost (as TOTAL_BOOKING_COST) for a team at a venue. Flag any potential

#booking that is going to exceed the venue maximum booking fee. (You can chose the team and the venue.)

#query has the variable pasted right into it for as an example which is dif from being a variable in python

4 is the over booking fee

SELECT

team.name AS team_name,

team.hourly_rate AS team_hourly_rate,

DATE_FORMAT(booking.start_datetime, '%d/%m/%Y %H:%i:%s') AS start_time,

DATE_FORMAT(booking.end_datetime, '%d/%m/%Y %H:%i:%s') AS end_time,

venue.max_booking_fee AS venue_max_fee,

TIMESTAMPDIFF(HOUR, booking.start_datetime, booking.end_datetime) AS duration_in_hours,

(TIMESTAMPDIFF(HOUR, booking.start_datetime, booking.end_datetime) * team.hourly_rate) AS total_booking_cost,

CASE

WHEN (TIMESTAMPDIFF(MINUTE, booking.start_datetime, booking.end_datetime) / 60.0 * team.hourly_rate) > venue.max_booking_fee

THEN 'OVER BUDGET'

ELSE 'WITHIN BUDGET'

```

        END AS booking_status
FROM booking
JOIN venue ON booking.venue_id = venue.venue_id
JOIN team_presentation ON booking.team_presentation_id =
team_presentation.team_presentation_id
JOIN team ON team_presentation.team_id = team.team_id
WHERE booking.venue_id = 1;

```

part_7.sql

#part 7

#A booking has a start time and end time, and each team has a fee in £ per hour. Write a query in Python to

#run a calculated field of the total event cost (as TOTAL_BOOKING_COST) for a team at a venue. Flag any potential

#booking that is going to exceed the venue maximum booking fee. (You can chose the team and the venue.)

#query has the variable pasted right into it for as an example which is dif from being a variable in python

4 is the over budget entry

SELECT

```

    team.name AS team_name,
    team.hourly_rate AS team_hourly_rate,

```

```

    DATE_FORMAT(booking.start_datetime, '%d/%m/%Y %H:%i:%s') AS
start_time,

```

```

    DATE_FORMAT(booking.end_datetime, '%d/%m/%Y %H:%i:%s') AS
end_time,

```

```

    venue.max_booking_fee AS venue_max_fee,

```

```

    TIMESTAMPDIFF(HOUR, booking.start_datetime,
booking.end_datetime) AS duration_in_hours,

```

```
(TIMESTAMPDIFF(HOUR, booking.start_datetime,
booking.end_datetime) * team.hourly_rate) AS total_booking_cost,
CASE
    WHEN (TIMESTAMPDIFF(MINUTE, booking.start_datetime,
booking.end_datetime) / 60.0 * team.hourly_rate) >
venue.max_booking_fee
    THEN 'OVER BUDGET'
    ELSE 'WITHIN BUDGET'
END AS booking_status
FROM booking
JOIN venue ON booking.venue_id = venue.venue_id
JOIN team_presentation ON booking.team_presentation_id =
team_presentation.team_presentation_id
JOIN team ON team_presentation.team_id = team.team_id
WHERE booking.venue_id = 1;
```

part_8.sql

#part 8

#Create a MySQL INSERT INTO query in Python, operated from the main menu. It needs to add a further 2 teams

#from the user's input, and add at least 2 members each per team (also from user input), ensuring any relevant

#connected tables are updated too.

```
INSERT INTO team (name, hourly_rate) VALUES ('The Paul Jackson
team', 500);
```

```
SET @last_team_id = LAST_INSERT_ID();
```

```
INSERT INTO member (name) VALUES ('Richard Hammond');
```

```
INSERT INTO team_has_member (team_id, member_id) VALUES  
(@last_team_id, LAST_INSERT_ID());
```

```
INSERT INTO member (name) VALUES ('James May');
```

```
INSERT INTO team_has_member (team_id, member_id) VALUES  
(@last_team_id, LAST_INSERT_ID());
```

```
COMMIT;
```

part_9.sql

```
SELECT * FROM team;
```

part_10.sql

```
SELECT  
    member.member_id,  
    member.name AS member_name,  
    member.email,  
    member.phone  
FROM member  
JOIN team_has_member ON member.member_id =  
team_has_member.member_id  
JOIN team ON team_has_member.team_id = team.team_id  
WHERE team.name = 'The Paul Jackson team';
```